



TAL Project

Chu Amélie 21206329
Tambellini Alan 21203966

September 16, 2026

Contents

1	Introduction	3
2	EDA	3
2.1	Presidents	3
2.2	Movies	4
2.3	Topic Modeling with LDA and LSA	6
2.4	Vocabulary Visualisation	10
3	Training and Evaluation process	11
3.1	Vocabulary size and training time	11
3.2	Cross-validation vs. single split	12
3.3	Preprocessing	13
3.4	N-grams	14
3.5	Vector size	15
3.6	Vectorizers and embeddings comparison	15
3.7	Classical models	16
3.8	Recurrent Neural Networks	17
3.9	Transformers	19
3.9.1	Presidents	19
3.9.2	Movies	20
3.10	Smoothing	22
4	Submissions	26
4.1	Presidents	26
4.2	Movies	28
5	Appendix	29
5.1	RNN	29

1 Introduction

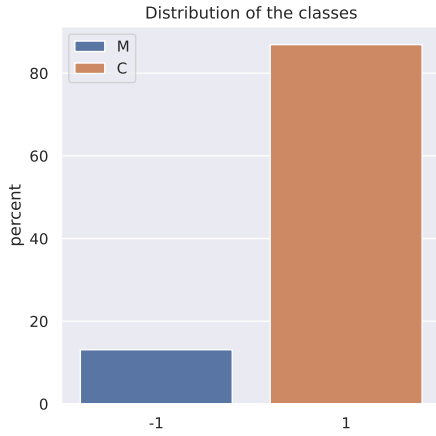
In this project, we address two text classification tasks. The first focuses on identifying the speaker, either Chirac or Mitterrand, using the **Presidents** dataset, which contains excerpts from their speeches. The second task is sentiment analysis on the **Movies** dataset, where reviews are labeled as positive or negative. Throughout this paper, we examine methods for text preprocessing, word representation, and model application.

2 EDA

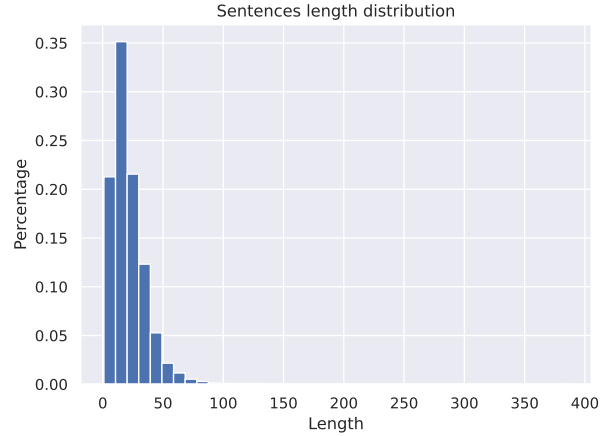
We first begin with an exploratory data analysis of both datasets.

2.1 Presidents

The Presidents dataset is in **french**. There are in total 57,413 sentences, of which 7,523 ($\sim 13\%$) correspond to Mitterrand, and 49,890 to Chirac. We note that for the metrics, we will account for the imbalance of the dataset. The vocabulary without stopwords totals 27,294 words.

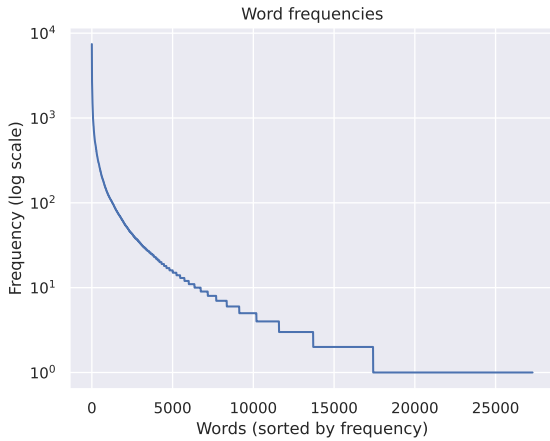


(a) Class distribution.

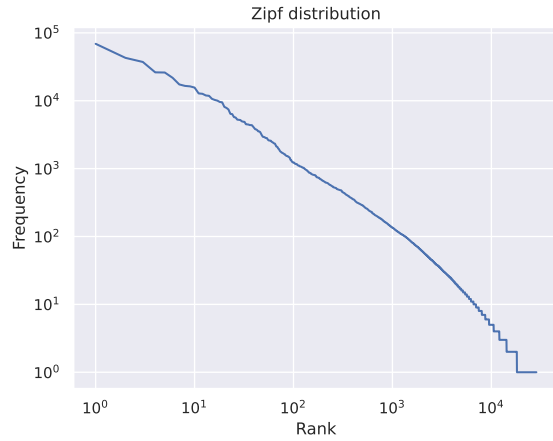


(b) Length distribution.

Figure 1: Distributions of Presidents.



(a) Word frequencies.



(b) Zipf law.

Figure 2: Word frequencies and rank.

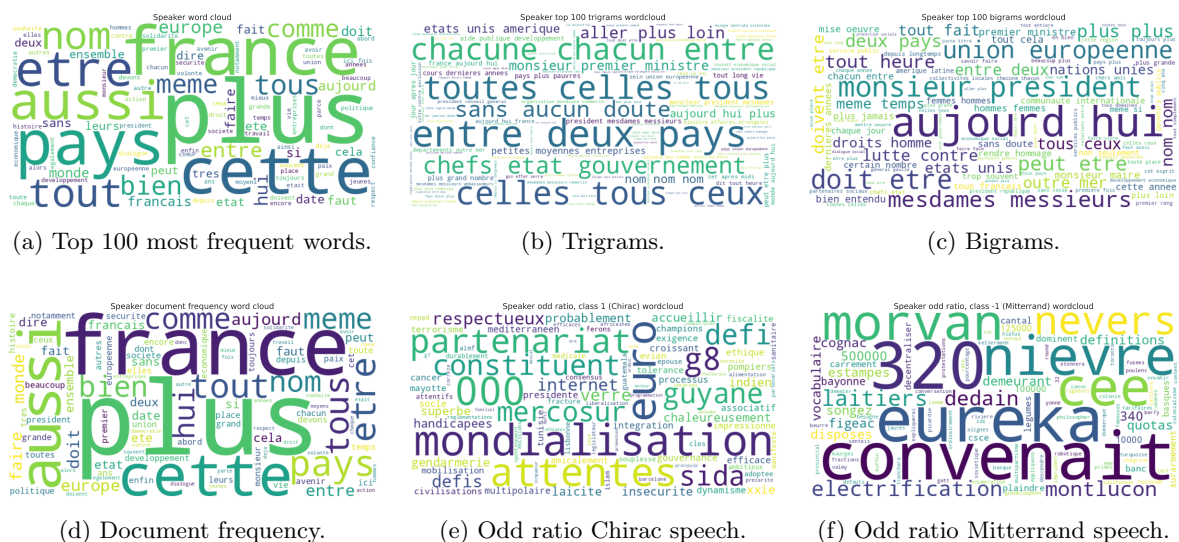


Figure 3: Top 100 words wordcloud.

It is worth noting that the Presidents dataset includes an identifier feature of form $\langle x : y \rangle$. We could potentially exploit this information for model predictions, however we chose not to explore this direction and instead focused solely on textual features.

2.2 Movies

The Movies dataset contains reviews in **english**. Each document correspond to one single review. There are 2000 reviews in total, with equal proportions between negative and positive reviews. Furthermore, the lengths of documents are approximately 10 times longer than in Presidents.

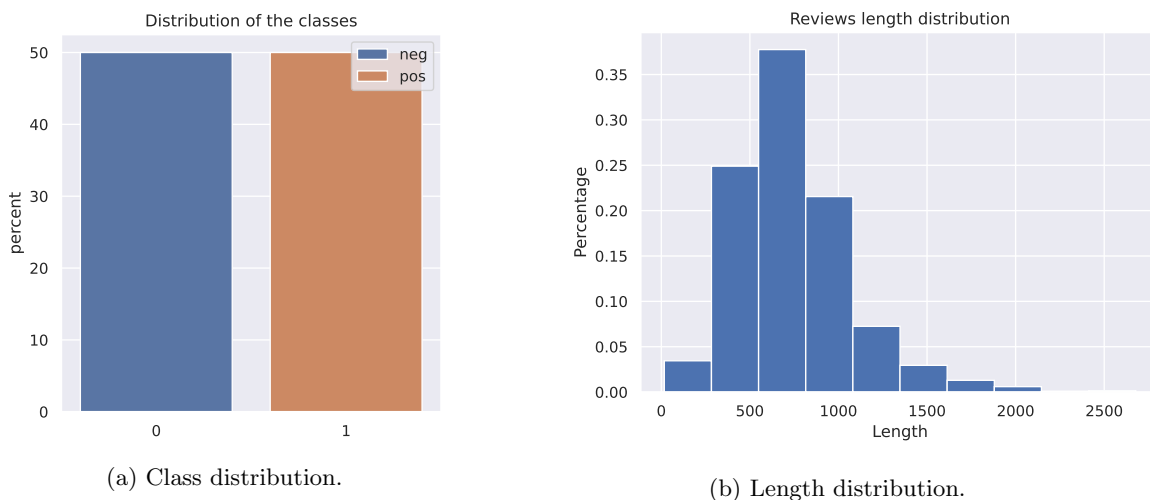


Figure 4: Distributions for Movies.

We do a simple preprocess where we remove punctuation, URLs, and accents, keeping everything in lowercase. We then look at the word frequencies and the vocabulary, which is of size 39,363 in total, and 39,058 if we remove stop words.

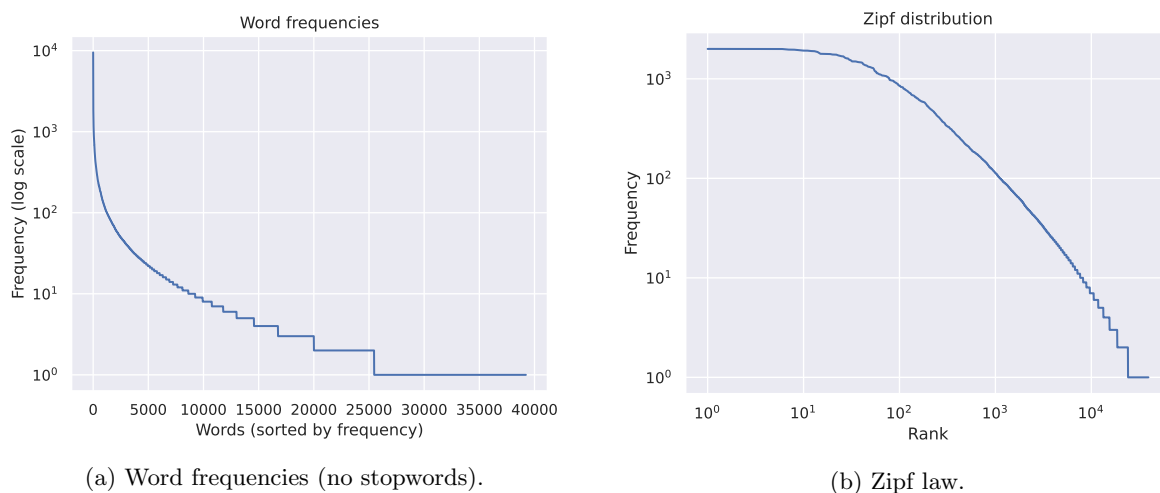


Figure 5: Word frequencies and rank.

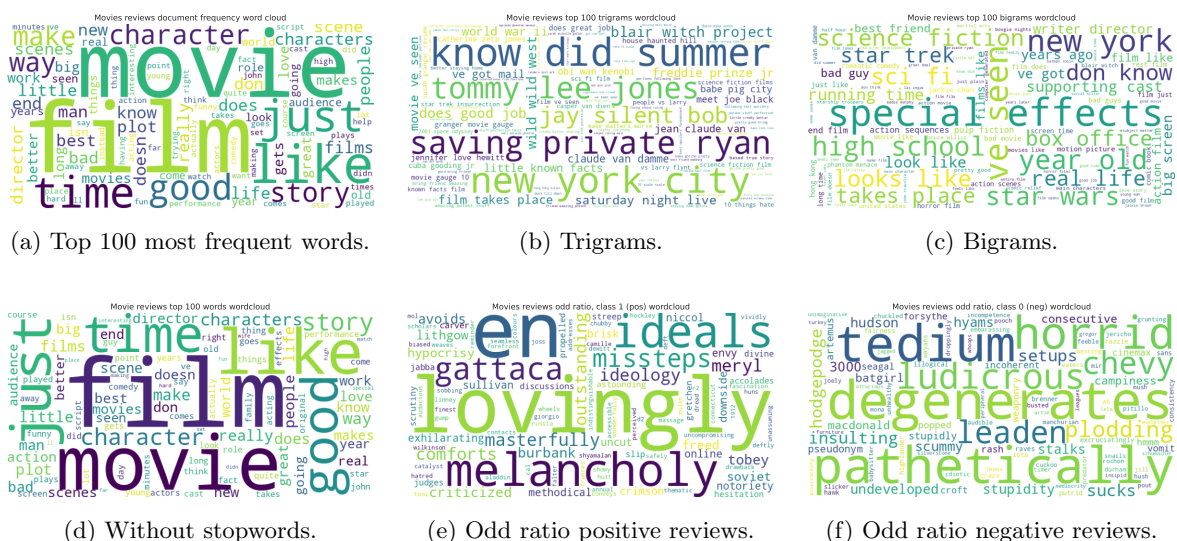


Figure 6: Word clouds for Movies.

2.3 Topic Modeling with LDA and LSA

To better understand the thematic structure of our texts, we can apply two topic-modeling approaches: Latent Semantic Analysis (LSA) and Latent Dirichlet Allocation (LDA). These methods project high-dimensional word-based representations into a low-dimensional topic space, which can help with classification tasks, clustering, and interpreting our visualizations.

Given a term-document matrix $X \in R^{V \times D}$ (with TF-IDF counts), LSA computes a truncated Singular Value Decomposition

$$X = U_k \Sigma_k V_k^\top, \quad (1)$$

where each document is represented by its corresponding row in $V_k \in R^{D \times k}$, encoding its position in the k -dimensional latent space.

Unlike LSA’s linear dimensionality reduction, LDA provides a probabilistic generative model that explicitly discovers interpretable topics as mixtures of documents, which we can use to do topic prevalence analysis that LSA cannot reveal. We can then visualize how documents cluster by speaker or sentiment, depending on the dataset, and to see if the classes in the topic space are separable.

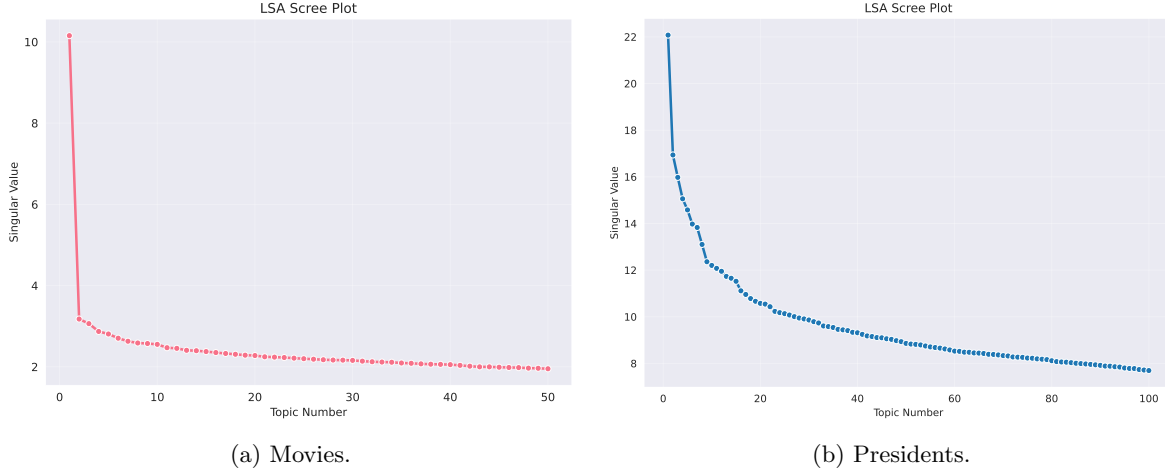


Figure 7: Scree plot of LSA.

The scree plots in Figure 7 show singular values decreasing rapidly after the first few topics, which indicates the first few topics capture the majority of the explained variance. To balance interpretability and information retention, we visually choose the number of components where the curve starts to flatten, which here is 20 for Movies and 40 for Presidents.

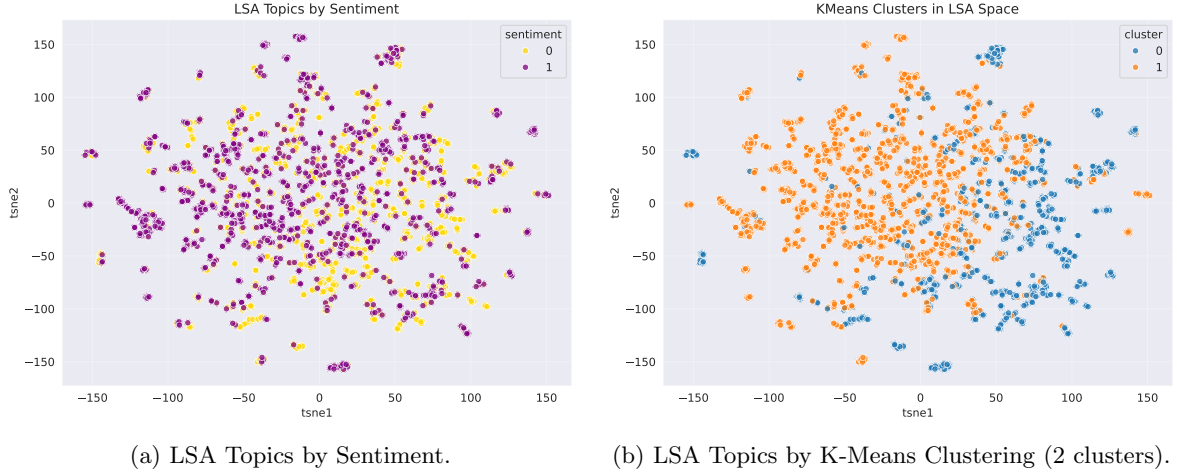


Figure 8: LSA Topic Space Analysis for Movies.

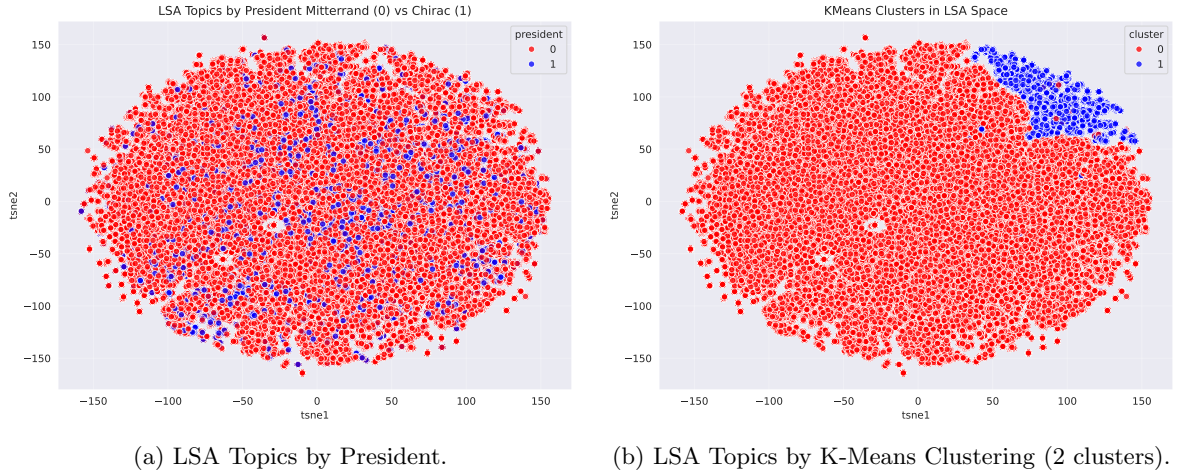


Figure 9: LSA Topic Space Analysis for Presidents.

Figures 8a and 9a display LSA topic vectors projected to 2D via t-SNE and colored by labels. We observe a lack of clear separability for both datasets. There are some small clusters in Movies, indicating some niche genres. However, the topics for Presidents are all grouped up into one dominant cluster, which could mean the topics are shared between the speakers. We conclude that, in the LSA topic space, reviews don't cluster strongly by sentiment, nor sentences by presidents.

Furthermore, K-Means applied to the LSA topic space does not reveal distinct clusters. For Movies, LSA with 50 components explains 6.8% of the TF-IDF variance, which is typical with high-dimensional sparse matrices where variance is spread across many dimensions. Therefore, while LSA captures thematic structure, the low explained variance and poor separability suggest that topical features aren't enough for classification.

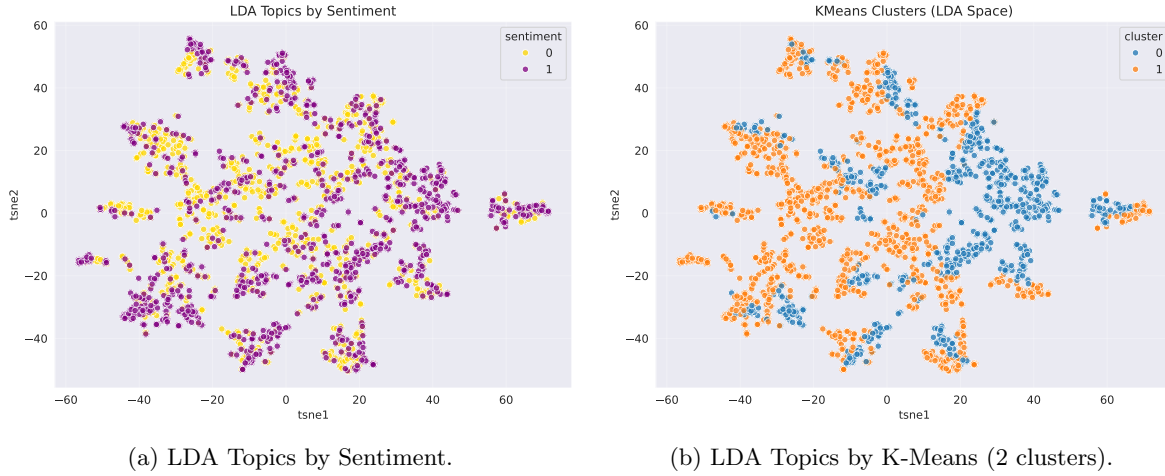


Figure 10: LDA Topic Space Analysis for Presidents.

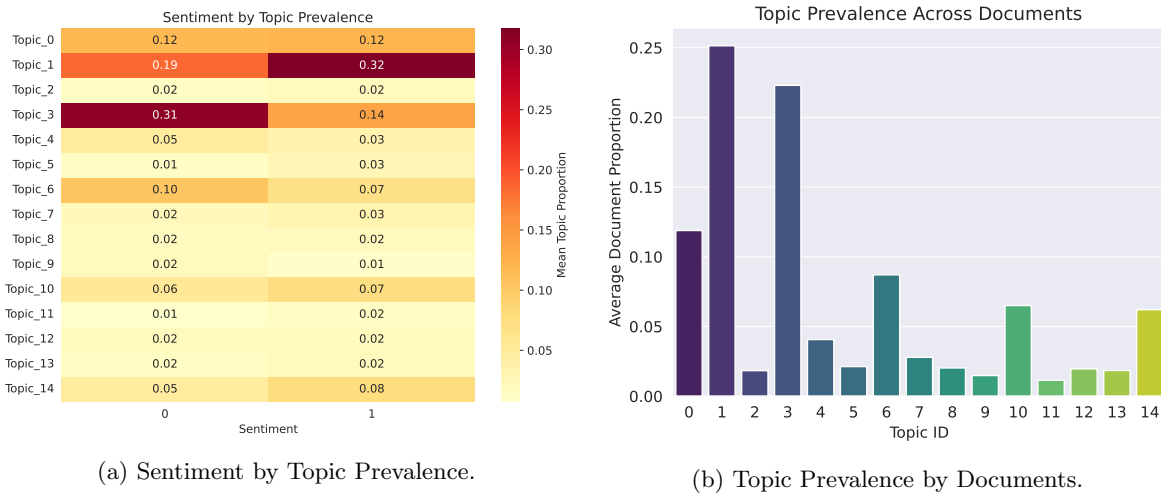


Figure 11: Topic Prevalence for Movies.

Compared to LSA, LDA topic projections in Figure 10a reveal much clearer structure. The probabilistic nature of LDA enables better capture of thematic mixtures compared to LSA’s linear dimensionality reduction. Furthermore, as we can see in Figure 11b, LDA modeling forms distinct genre clusters such as Topic 2 (“disney”, “tarzan”, “animation”) and 13 (“jackie chan”, “hong kong”, “fight”), while general review topics (1, 3) remain central.

Lastly, Figure 11a shows some distinct sentiment separation, with Topic 3 (“movie”, “bad”, “good”, “really”) strongly favoring negative reviews while Topic 1 (“film”, “life”, “story”, “love”) leans positive.

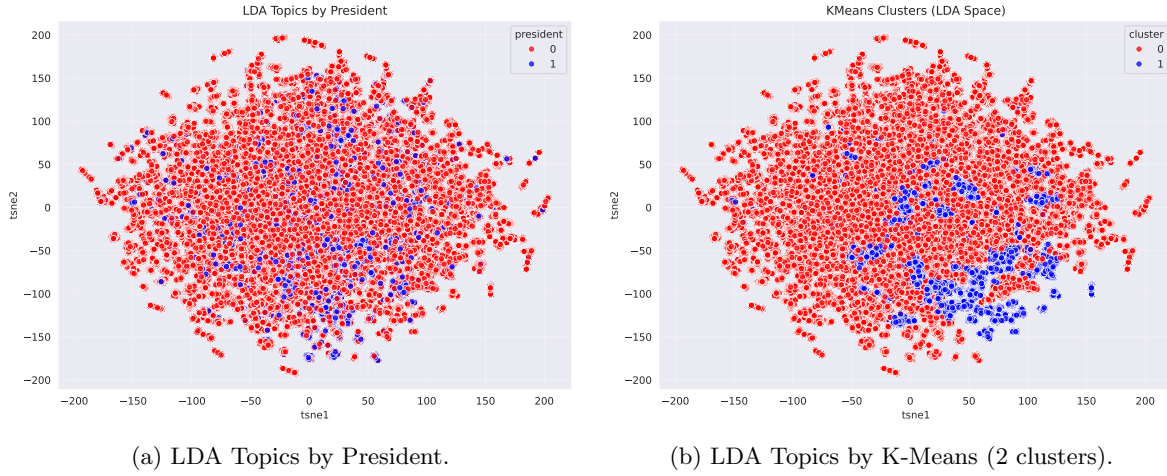


Figure 12: LDA Topic Space Analysis for Presidents.

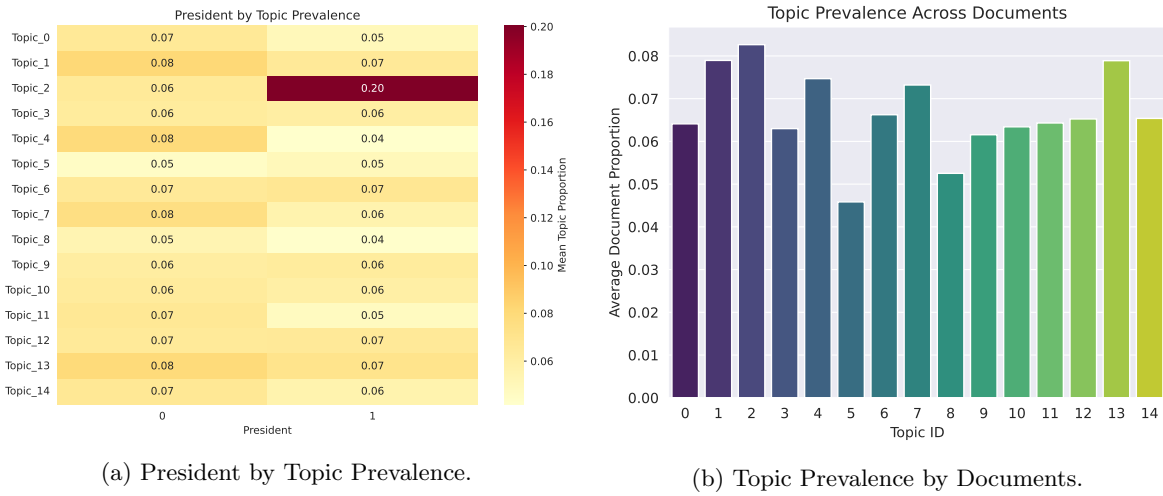
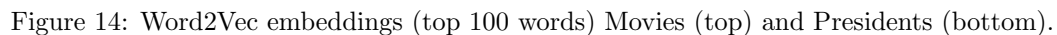


Figure 13: Topic Prevalence for Presidents.

For Presidents, we can see that LDA doesn't improve clustering capability. Figure 13 reveals that topic prevalence is distributed almost equally. This indicates the dataset consists of many overlapping themes, which is to be expected for political speakers. From this analysis, we can predict that the classifying task with the Presidents dataset might be the harder task of the two.

To complement topic modeling, we train Word2Vec skip-gram embeddings on the corpus (100 dimensions, window=5) and visualize the 200 most frequent words via t-SNE (Figure 14). Unlike LDA/LSA’s document-level topics, Word2Vec reveals fine-grained lexical relationships within the vocabulary.



10

3 Training and Evaluation process

In this section, we analyze the impact of different models and textual representations on the performance of our classification tasks. Given the large number of possible configurations, we adopt a progressive approach, starting with simpler setups and gradually increasing complexity. The dataset is consistently split into training and testing subsets to ensure reliable evaluation. Our experiments are structured around several key factors:

Factors	Choices
Preprocessing	Lemmatization/stemming, lowercasing, stopwords removal, accent removal, punctuation removal
Vectorization	Bag-of-Words (BoW), TF-IDF, embeddings trained on the training set, pretrained embeddings, Transformer Encoder model
Models	Logistic Regression, Naïve Bayes, Support Vector Machines (SVM), Random Forest (RF), Recurrent Neural Networks (RNN), Transformers
Hyperparameters	Vocabulary size, embedding dimension
Metrics	F1, AUC, AP, Precision, Recall, Accuracy

Table 1: Experimental factors

For the vectorization, we also included to experiment with a pre-trained transformer encoder model. We use the Multilingual E5¹ base model with 86M parameters, chosen among models under 1B parameters on the MTEB Hugging Face² leaderboard which benchmarks embedding models.

Before evaluating a wider range of models, we conduct preliminary experiments to determine appropriate parameter settings. For the Presidents dataset, model performance is assessed using macro F1 score, Area Under the Curve (AUC), and Average Precision (AP), which are more informative than accuracy due to class imbalance. For the Movies dataset, we report all of the metrics above.

Each metric captures a different aspect of performance: F1 balances precision and recall, AUC measures the model’s ability to rank positive instances higher than negatives, AP summarizes precision-recall trade-offs, and Accuracy, Precision, and Recall provide complementary insights for balanced evaluation.

3.1 Vocabulary size and training time

For Bag-of-Words (BoW) and TF-IDF representations, the feature space is determined by the vocabulary size. Larger vocabularies increase computational cost, so we aim to find a balance between efficiency and performance. This choice is informed by the word frequency distributions shown in Figures 2a and 5a. We experiment with vocabularies of up to 10,000 words for the Presidents dataset and 20,000 words for Movies.

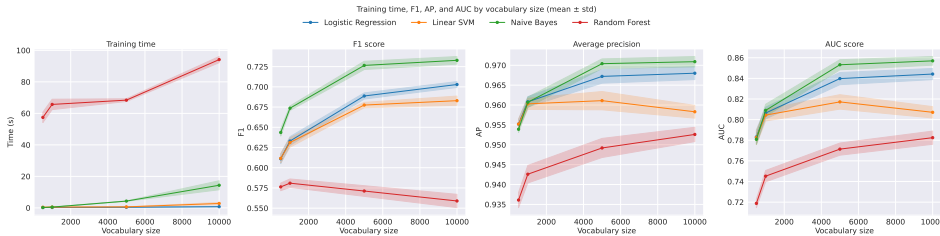


Figure 15: Training time by vocabulary size for Presidents.

¹ <https://huggingface.co/intfloat/multilingual-e5-base> ² <https://huggingface.co/spaces/mteb/leaderboard>

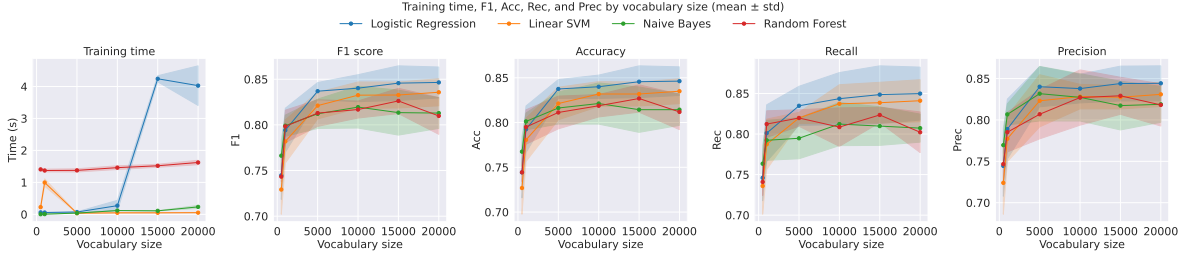


Figure 16: Training time by vocabulary size for Movies.

Figures 15 and 16 show the training times of various simple models across different vocabulary sizes. Preprocessing included lowercasing, punctuation removal, and stopword elimination, and the resulting text was vectorized using BoW. Performance scores were averaged over five runs with cross-validation. From these results, a vocabulary size of 5,000 words provides a good trade-off between computational cost and model performance for both datasets.

3.2 Cross-validation vs. single split

While cross-validation is more computationally intensive than a single train-test split, it generally yields more stable estimates of model performance. To quantify this difference, we compared 5-fold cross-validation with single splits repeated 20 times using logistic regression. As shown in Figures 17 and 18, the results were very similar, with variance on the order of 10^{-2} . For the Presidents dataset, this low variance is likely due to the stratified splits that account for class imbalance.

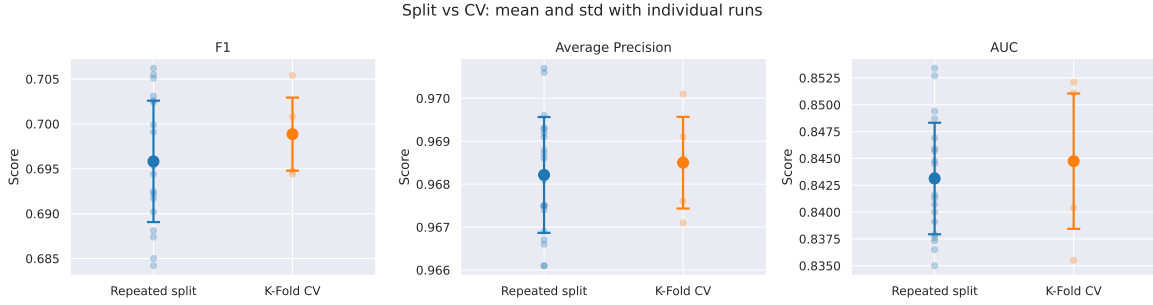


Figure 17: Score and variance for cross-validation and split for Presidents.

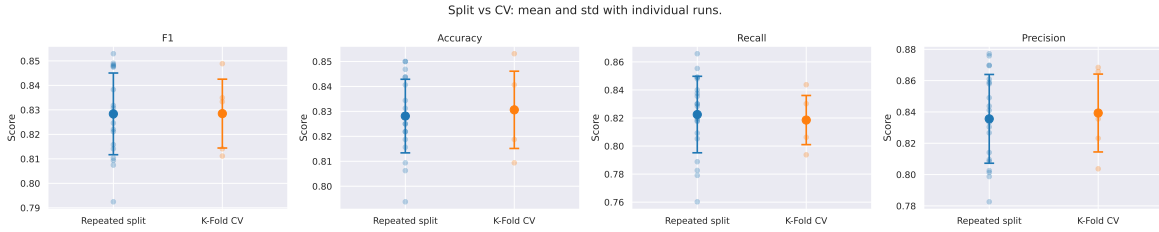


Figure 18: Score and variance for cross-validation and split for Movies.

Although logistic regression showed only minor differences, other models might exhibit higher variability. Therefore, we still adopt stratified 5-fold cross-validation for all subsequent experiments. This approach preserves class ratios in each fold and provides more reliable and comparable performance estimates, particularly for datasets like Presidents with class imbalance.

3.3 Preprocessing

Since our data consists of text, preprocessing is required before applying models. There are several possible preprocessing steps, such as removing punctuation, converting text to lowercase, and applying lemmatization or stemming to reduce variations due to grammar and conjugation. For french text, handling accents is also considered. These different options are combined to define multiple preprocessing pipelines.

To evaluate their impact, we use logistic regression with both BoW and TF-IDF vectorization, in order to identify the most effective pipeline without testing every combination across all models.

Name	Preprocessing
Prep_1	No preprocessing, raw text
Prep_2	Remove punctuation & stopwords
Prep_3	Prep_2 + lower case
Prep_4	Prep_3 + remove accents
Prep_5	Prep_3 + lemmatisation
Prep_6	Prep_3 + stemming

Table 2: Different preprocessings

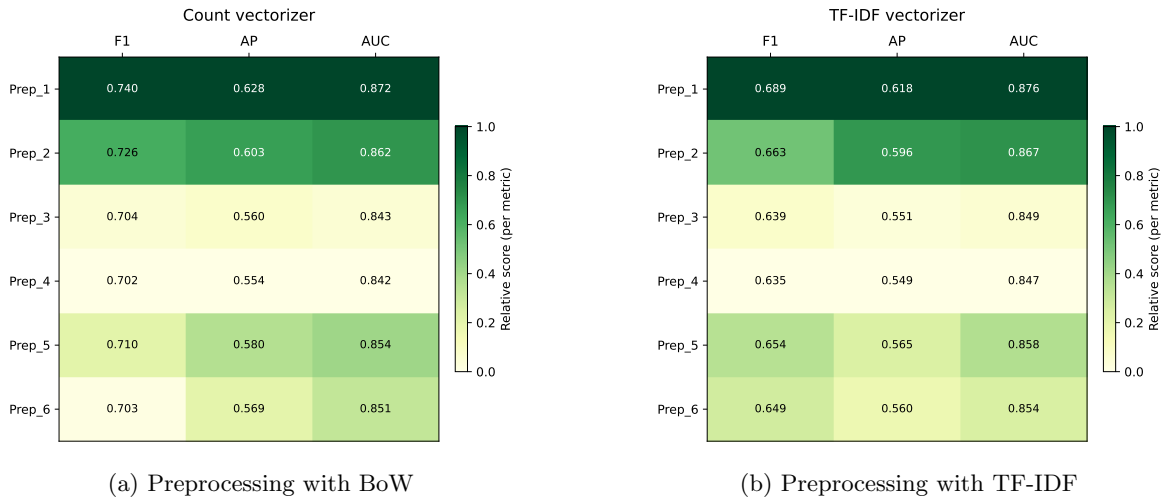


Figure 19: Preprocessing scores with Logistic Regression for Presidents.

On the Presidents dataset, the slightly lower scores observed with TF-IDF may be attributed to the relatively short length of the sentences, as shown in Figure 1b. As illustrated in Figure 19, BoW and TF-IDF achieve the best performance on raw text. Lowercasing slightly reduced the score, while lemmatization led to a modest improvement.

This suggests that sentence structure may play a key role in distinguishing the two speakers, including aspects such as tense, which we will explore later using Part-of-Speech (POS) tagging.

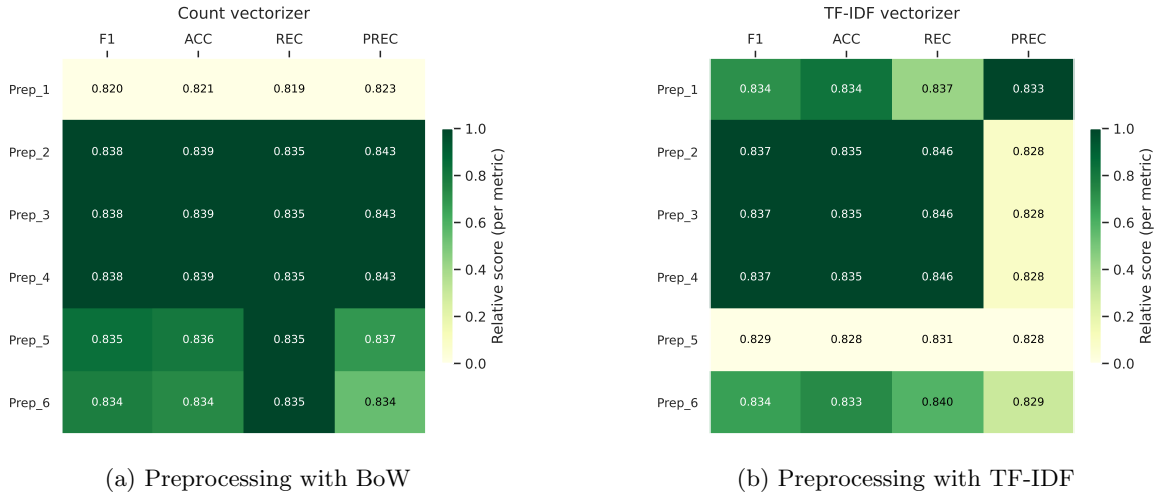


Figure 20: Preprocessing scores with logistic regression for Movies.

On the Movies dataset, we can see that the results are very similar using a BoW, but more varied with TD-IDF. We notice that Prep_2 and Prep_3 have similar score: that comes from the fact that in the train set, the reviews are already preprocessed: it is already lower cased, hence the similar results. Prep_4 appears in the evaluation but shouldn't be considered for Movies as there is no accent in the reviews. We also note that while the train set has already been preprocessed, it is not the case for the true test set that we have.

We then fix some pipelines for subsequent experiments with other models, acknowledging that the optimal preprocessing may vary by architecture, but assuming that the ranking observed with logistic regression is indicative of a general text-normalization choice.

3.4 N-grams

We experiment with n-grams up to trigrams, applying minimal preprocessing (Prep_2) and extracting n-grams directly from the text. These features are evaluated using logistic regression with 5-fold cross-validation. As seen in Figure 21, the results indicate little improvement over using unigrams alone.

This may be because, when limiting the vocabulary to the first 5,000 words, for the (1,2) and (1,3) n-gram ranges, 80% of the vocabulary consists of unigrams for the Presidents dataset, and up to 90% for the Movies dataset. Based on these results, we can subsequently restrict experiments to unigrams and bigrams.

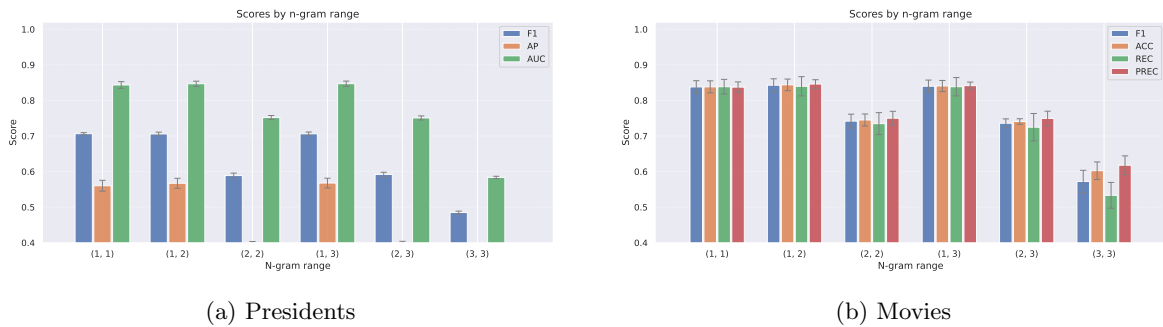


Figure 21: Scores by n-gram range for Presidents and Movies datasets.

3.5 Vector size

Another approach to text vectorization is to use embedding models, either pretrained or trained on our own data, such as Word2Vec, FastText, or Doc2Vec. The embedding dimensionality can be set up to 300. As shown in Figure 22, when trained on the training data, the evaluation metrics vary only slightly between dimensions 200 and 300. Therefore, a dimensionality of 200 can be safely chosen, as the performance difference compared to 300 is minimal.

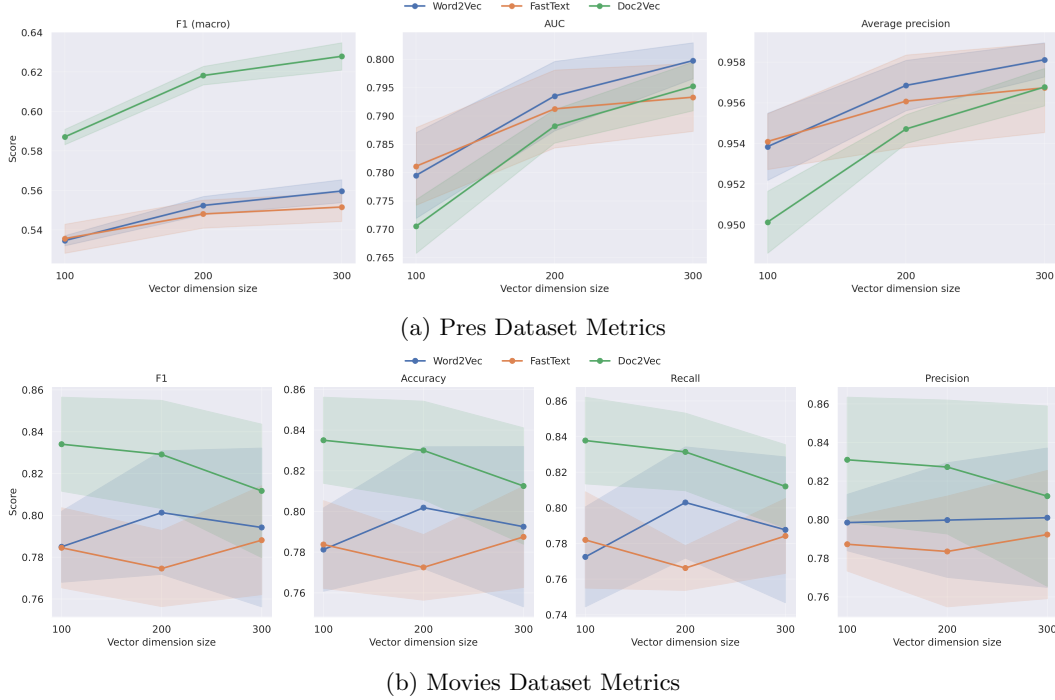


Figure 22: Comparison of vector size metrics across datasets.

For pretrained embeddings, we retained the original size of 300, since reducing the dimension only requires truncating the vectors, and do not add more computation time. This choice may partly explain the slightly lower performance observed for embeddings trained on our own data, which have a lower dimensionality.

We also experiment with a pretrained transformer-based encoder, the multilingual E5-base model, which supports multiple languages. For this encoder, we use embeddings of size 768 to generate sentence-level representations, evaluating the performance of the entire sentence directly rather than word by word. The model initially computes embeddings at its full dimensionality before truncation to the chosen size, so we decided to keep the full dimensionality.

3.6 Vectorizers and embeddings comparison

With a simple preprocessing (Prep_2), we evaluate a logistic regression model with different text vectorization methods. Based on these initial results, we can then select the most promising representations for further experiments.

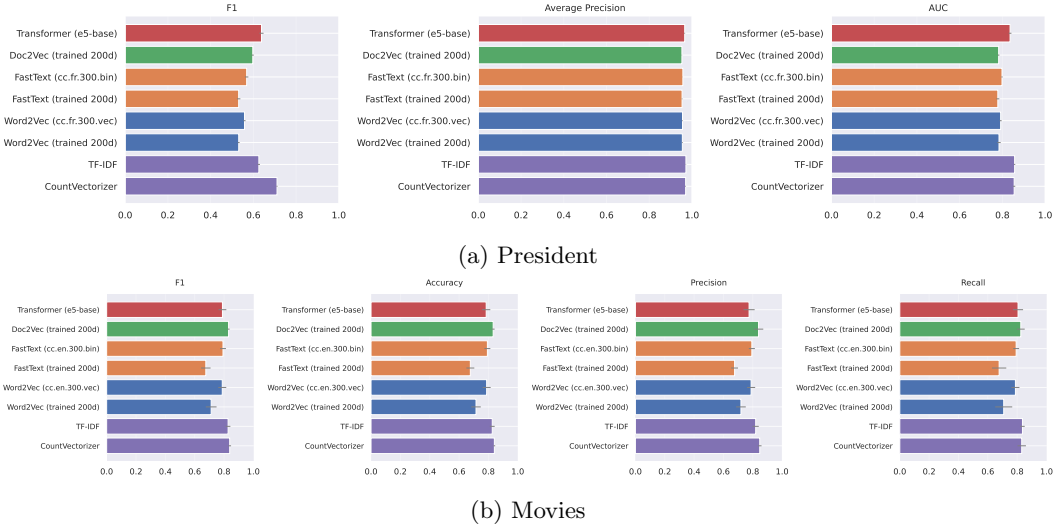


Figure 23: Different vectorization methods performance across datasets.

Figure 23 shows the best performance for the Presidents dataset using BoW, TF-IDF, and the transformer encoder, and similarly for the Movies dataset, including Doc2Vec.

3.7 Classical models

Using classical models with predefined parameters from previous experiments, we evaluate their performance on both cross-validation and test sets for both datasets. The models are tested with unigram and bigram features, using either TF-IDF or BoW representations. Figure 24 shows the top 10 models ranked by F1 score. For each dataset, we applied the preprocessing pipeline that was found to be optimal, as described in [Preprocessing](#).

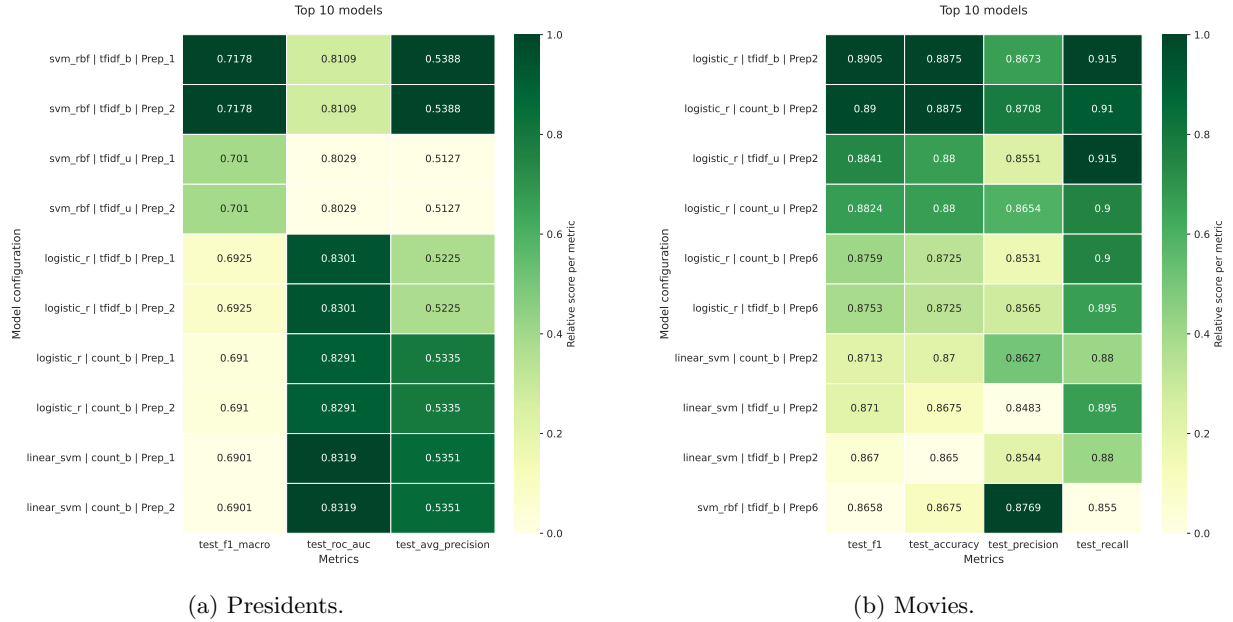


Figure 24: Top 10 model combinations ranked by F1 score.

On the left the combinations are defined by : the model | BoW or TF-IDF and unigrams or bigrams | the preprocessing used.

3.8 Recurrent Neural Networks

To model sequential inputs with memory, we consider Recurrent Neural Networks (RNNs), specifically using the **LSTM** and **GRU** architectures to capture long-term dependencies. We evaluate this model with different text vectorization methods, and we also experiment here with the idea of POS tags to capture sentence structure, extracting for the verbs tags, the tense of the verbs.

Our baseline architecture employs an embedding layer with a fixed vocabulary size of 10,000. We use an embedding dimension of 768 for the encoder-based model, and dimension 32 to all other variants. We expanded the vocabulary size from 5,000 to 10,000 as the embedding dimension remains independent of the input feature space.

These embeddings are processed by a 2-layer LSTM or GRU bidirectional, with a hidden dimension of 64. The models are optimized using cross-entropy loss, with class weights for the Presidents dataset, and trained for a maximum of 50 epochs. To prevent overfitting, we implement an early stopping mechanism: training is terminated if the validation F1-score fails to improve for 10 consecutive epochs. The final model parameters are determined by the peak validation F1-score and is then evaluated on the test set. The resulting performance metrics for each configuration are reported below.

Since the GRU and LSTM models yielded similar results, only the higher-performing GRU configurations are discussed in the main text. The full results for LSTM models are available in [Appendix](#).

Test scores	F1	AP	AUC
GRU POS tag	0.38	0.37	0.74
GRU BoW	0.62	0.32	0.71
GRU TF-IDF	0.62	0.33	0.71
GRU encoder-based	0.50	0.51	0.84

Table 3: President GRU test scores.

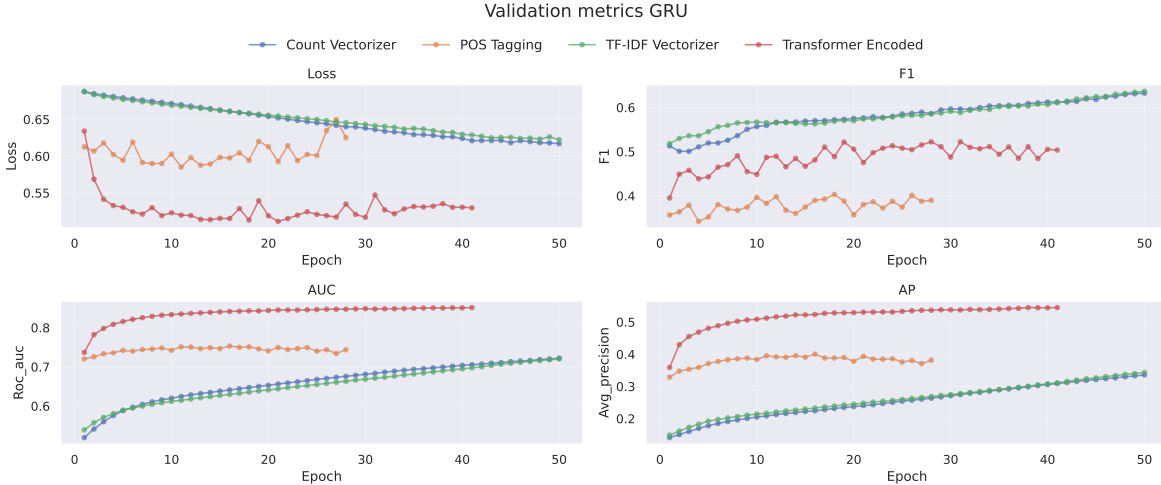


Figure 25: President GRU metrics over epochs

Table 3 shows that for the Presidents dataset, the encoder-based model achieved the best AP and AUC scores. However, its relatively low F1-score compared to BoW or TF-IDF suggests that the default 0.5 classification threshold is not optimal. On the other hand, the POS tag model yields the lowest F1-score, indicating that contrary to our hypothesis, syntactic structure alone is insufficient for effective classification without semantic context.

Figure 25 illustrates the convergence profiles of the four models. Both the encoder-based and POS-tag architectures show rapid convergence, reaching a stable loss minimum within 10 epochs. In contrary, the TF-IDF and BoW do not appear to have converged by epoch 50, suggesting that these models require more training to optimize their sparser feature sets.

Test scores	F1	Accuracy	AUC
GRU POS tag	0.67	0.50	0.56
GRU BoW	0.64	0.50	0.48
GRU TF-IDF	0.54	0.54	0.53
GRU encoder-based	0.79	0.78	0.86

Table 4: Movies GRU test scores

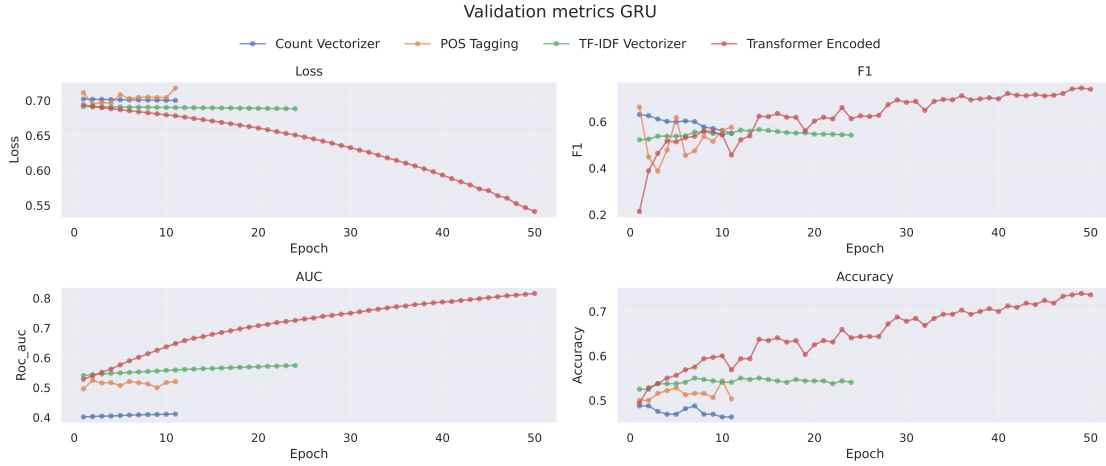


Figure 26: Movies GRU metrics over epochs

Similar performance trends are observed with the Movies dataset in Table 4. For the first three configurations, both AUC and accuracy scores are near 0.5, indicating that the models fail to capture meaningful underlying patterns and perform no better than a random classifier. The encoder-based model shows the best performance, and appears to have not converged yet. Training the model over more epochs may yield further performance gains, so we ran the model for 50 more epochs:

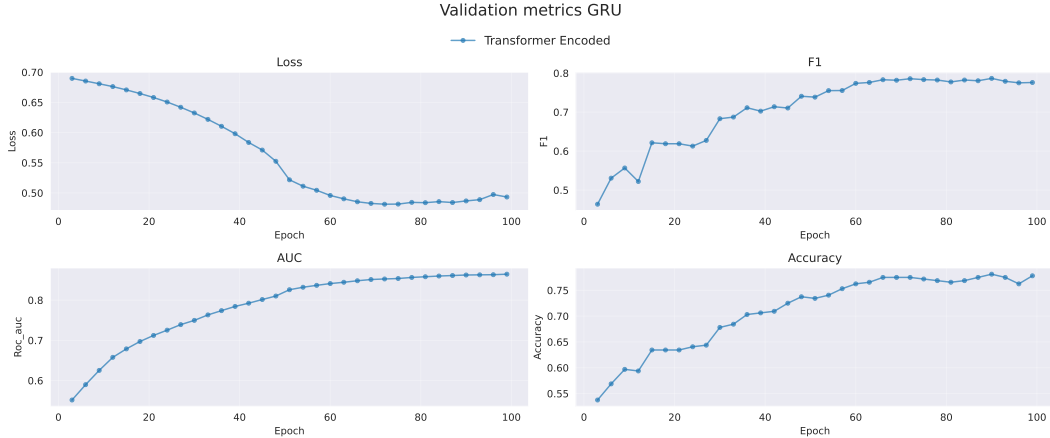


Figure 27: Movies GRU encoder-based metrics over epochs

We can see in Figure 27 that after 80 epochs the model stabilizes, with final test results of the GRU encoder-based model: F1 = 0.81, Accuracy = 0.81, and AUC = 0.89. To better capture intricate semantic dependencies in our data, we move onto Transformer-based architectures.

3.9 Transformers

Transformers are among the state-of-the-art models in natural language processing. We selected widely recognized transformer-based models for our tasks:

- for Presidents dataset, we used CamemBERT³ and CamemBERTa⁴
- for Movies dataset, we used BERT⁵ based and Facebook’s roBERTa⁶

We first conducted experiments on the Presidents dataset using a standardized configuration. The models were trained for up to 20 epochs with early stopping patience set to 5, and a maximum sequence length of 128 tokens. We used a learning rate of $2e^{-5}$, a batch size of 8, and gradient accumulation over 2 steps. We used a weighted loss for the class imbalance. All model parameters remained unfrozen, enabling full fine-tuning of the weights.

Training was carried out on the training and validation sets, while evaluation was performed on the test set. Early stopping was applied to select the model achieving the highest validation F1 score within the 20 training epochs.

3.9.1 Presidents

	F1 test	AP test	AUC test
CamemBERT	0.81	0.75	0.92
CamemBERTa	0.81	0.76	0.93

Table 5: Transformers scores Presidents.

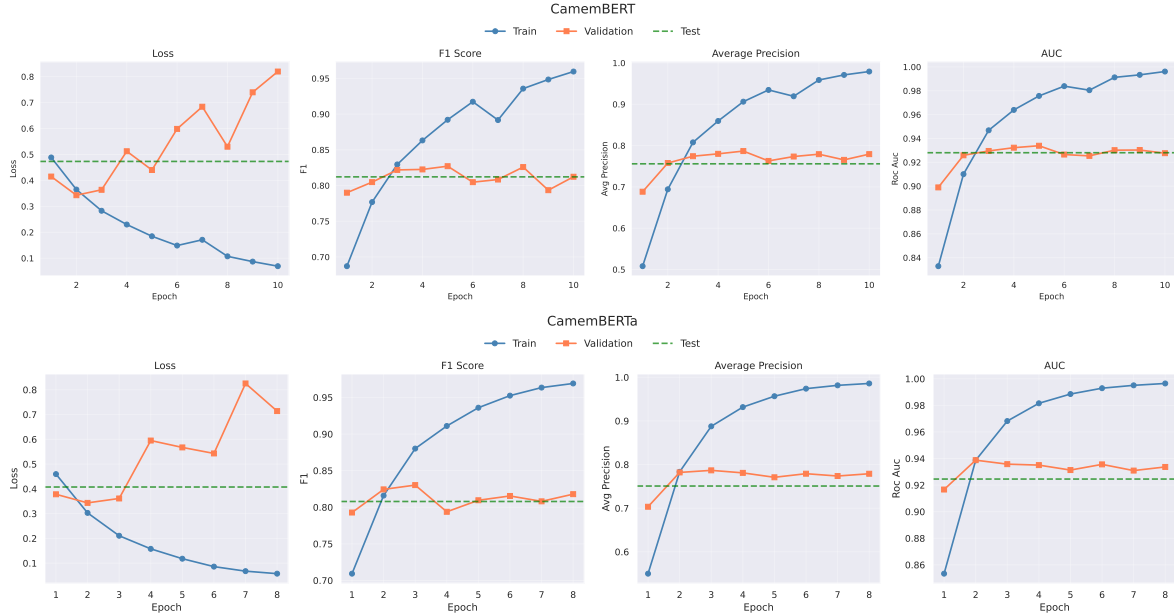


Figure 28: CamemBERT (top) and CamemBERTa (bottom) metrics over epochs Presidents.

³ <https://huggingface.co/almanach/camembert-base>

⁴ <https://huggingface.co/almanach/camemberta-base>

⁵ <https://huggingface.co/google-bert/bert-base-cased>

⁶ <https://huggingface.co/FacebookAI/roberta-base>

We can see for both models signs of overfitting after 3 epochs in Figure 28. Therefore, we shift our focus toward optimizing the fine-tuning process. Testing all different combinations is not possible due to the computation time. To try to find the best hyperparameters, we can adopt a sequential optimization strategy: we tune one hyperparameter at a time and fix it with the best-performing F1 score on the test. Due to the hardware constraints and the large size of the dataset, we will only be able to test a few fine-tuned models and not explore many possible variations.

Model	Token length	Batch size	LR	F1 test	AP test	AUC test
CamemBERTa (frozen) ⁷	128	8	$2e^{-5}$	0.71	0.52	0.85
CamemBERTa	128	16	$2e^{-5}$	0.81	0.74	0.93

Table 6: Some fine-tuned transformers scores Presidents.

3.9.2 Movies

For the Movies dataset, we first used the same base configuration as Presidents (except without weights for the loss).

	F1 test	Accuracy test	Precision test	Recall test
BERT	0.66	0.50	0.50	0.97
RoBERTa	0.75	0.73	0.70	0.81

Table 7: Transformers scores Movies.

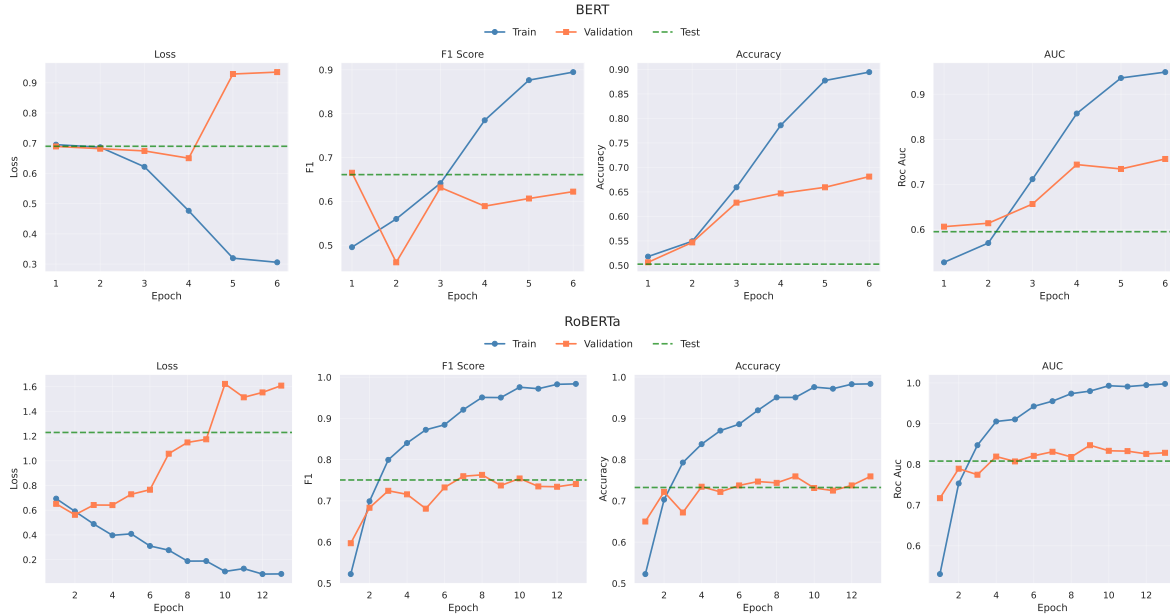


Figure 29: BERT (top) and RoBERTa (bottom) metrics over epochs Movies.

As seen in Table 7, RoBERTa outperforms BERT since it is a bigger model with a stronger pretrained encoder and longer context window. However, BERT seems to be more keen on predicting the positive class, which increases recall but lowers precision and accuracy. RoBERTa is slightly more conservative and better calibrated, so it makes fewer false positives and thus achieves a better balance between precision and recall.

⁷ Here we froze the weights, only modifying the classifier head

The validation curves in Figure 29 show that both models learn quickly but start overfitting very quickly, just like in Presidents. This is to be expected because Movies is a relatively small dataset of 2000 reviews.

We then move on to finetuning RoBERTa. After manually exploring some parameters, we discovered that a max length of 512 optimizes the scores really well. This is due to the length of the reviews being much longer than the sentences in the Presidents (comparing Figures 1b and 4b). Following that, we further fine-tuned that specific model to maximize the scores.

Model	Token length	Batch size	LR	F1 test	Accuracy test	Recall test
RoBERTaV1	512	8	$2e^{-5}$	0.89	0.89	0.90
RoBERTaV2	512	8	$2e^{-6}$	0.90	0.90	0.91
RoBERTaV3	512	8	$2e^{-5}$	0.90	0.90	0.94
RoBERTaV4	512	8	$2e^{-5}$	0.88	0.88	0.89
RoBERTaV5	256	16	$2e^{-6}$	0.66	0.5	1

Table 8: Transformers scores Movies, fine-tuning

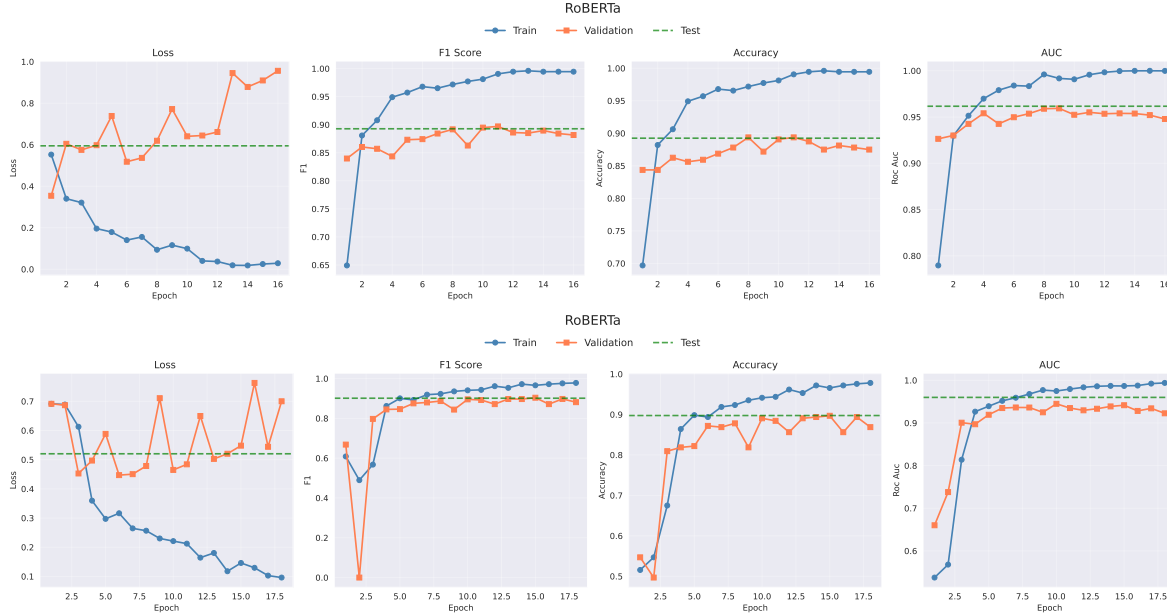


Figure 30: RoBERTaV1 (top) and RoBERTaV3 (bottom) metrics over epochs Movies.

RoBERTaV1, as said previously, starts off strong, but the validation loss increases after only a few epochs, meaning it still overfits quickly. To improve generalization, we tested stronger regularization, different learning rates, and modified training schedules. RoBERTaV2 used a lower learning rate to have a more gradual weight update, improving the score by a small margin.

RoBERTaV3 introduced classifier dropout (0.2) with a stronger weight decay (0.05), maintaining F1 while significantly boosting recall, meaning the dropout successfully regularized the decision boundary without sacrificing sensitivity to the positive class.

RoBERTaV4 tested single-step gradient updates to reduce gradient noise, but this destabilized convergence, indicating the original accumulation strategy better handled the small dataset’s gradient variance. Finally, RoBERTaV5 used shorter sequences (256 tokens) with larger batches, but the training failed due to context loss.

This confirms that 512 token inputs is optimal and classifier dropout with moderate weight decay is a good way to prevent overfitting.

3.10 Smoothing

For the Presidents dataset only, one interesting point is the consecutiveness of the labels, as shown in Figure 32. We could try to use a sliding window to smooth and ameliorate the probabilities predictions of models, as a post-processing part.

To preserve the sequential structure of the labels, we use a time-series split, which allows both cross-validation and a train-test split while keeping consecutive blocks together and avoiding shuffling. We used logistic regression and BoW to test the smoothing of the probabilities.

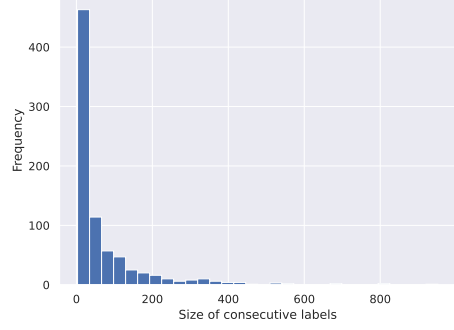


Figure 31: Distribution of lengths of consecutive label blocks

The experiment explores three different smoothing methods and their parameters, including the alpha for exponential smoothing, and the window size and standard deviation for the moving average methods. Considering the distribution of continuous label lengths shown in Figure 31, we test window sizes of up to 25 (the median is 26). To select the best smoothing method, we evaluate its effect on the probabilities.

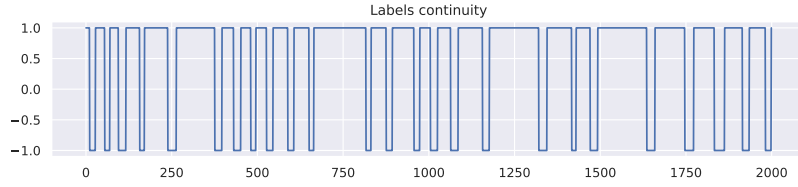


Figure 32: Continuity of labels across the dataset on 2000 samples

Sequential averaging smooths each probability using the previous smoothed value. The parameter α controls the weight given to the current probability versus the previous smoothed probability:

$$P_t^{\text{smooth}} = \alpha P_t + (1 - \alpha) P_{t-1}^{\text{smooth}}, \quad t = 1, 2, \dots, T, \quad P_0^{\text{smooth}} = P_0$$

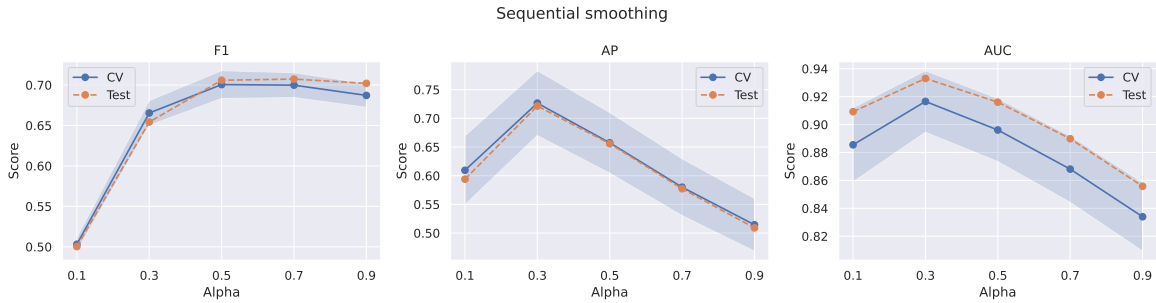


Figure 33: Scores for sequential averaging with varying α

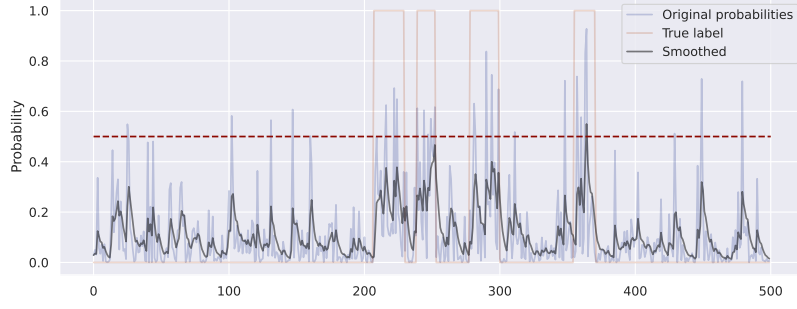


Figure 34: Sequential averaging with $\alpha = 0.5$ on 500 samples

This smoothing is not adapted to our task, as we need a bigger window and to take into account next probabilities too.

In the **averaging window** method, each probability is replaced by the mean of probabilities in a symmetric window of size w around it:

$$P_t^{\text{smooth}} = \frac{1}{w} \sum_{i=-k}^k P_{t+i}, \quad k = \frac{w-1}{2}$$

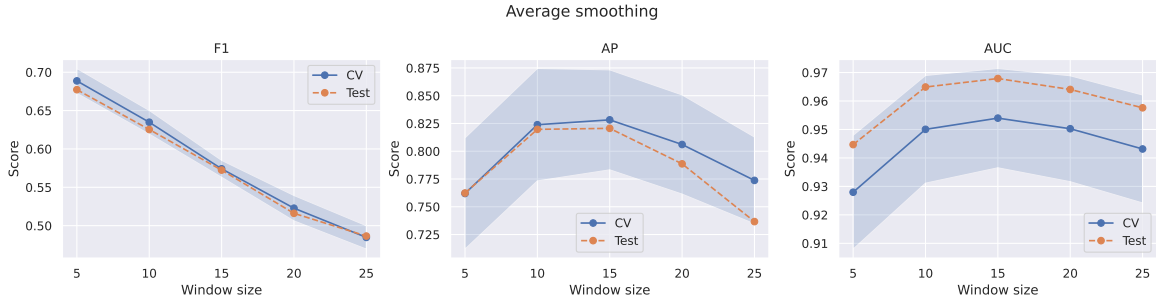


Figure 35: Scores for simple averaging over a window of size

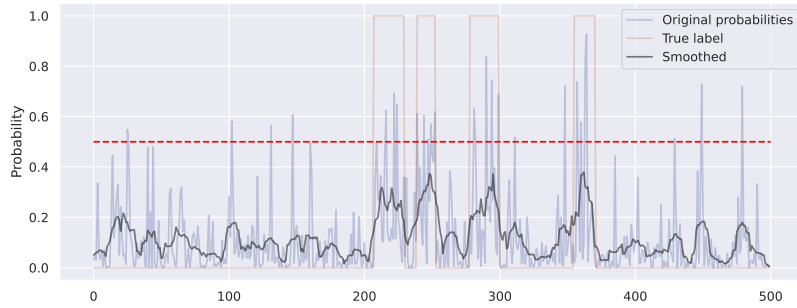


Figure 36: Averaging window with window size = 10 on 500 samples

For the simple averaging window, increasing the window size leads to a decrease in performance scores. This can be explained by the fact that larger windows incorporate more neighboring points from the different classes, and all points are weighted equally.

Gaussian averaging is similar to the simple averaging window but assigns higher weights to points closer to the current one. The weights are determined by a Gaussian kernel with standard deviation σ .

A smaller σ results in a narrower kernel, giving most of the weight to points very close to the current one, while a larger σ produces a wider kernel, distributing the weights more evenly across the window.

$$P_t^{\text{smooth}} = \sum_{i=-k}^k w_i P_{t+i}, \quad w_i = \frac{\exp\left(-\frac{i^2}{2\sigma^2}\right)}{\sum_{j=-k}^k \exp\left(-\frac{j^2}{2\sigma^2}\right)}, \quad k = \frac{w-1}{2}$$

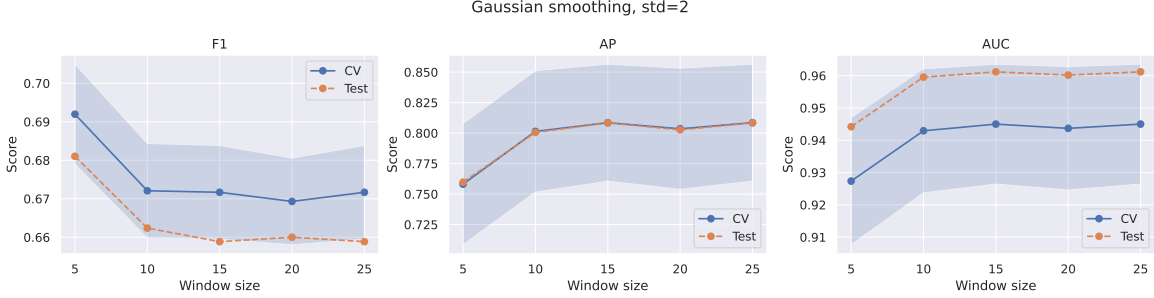


Figure 37: Scores for Gaussian averaging, $\sigma = 2$

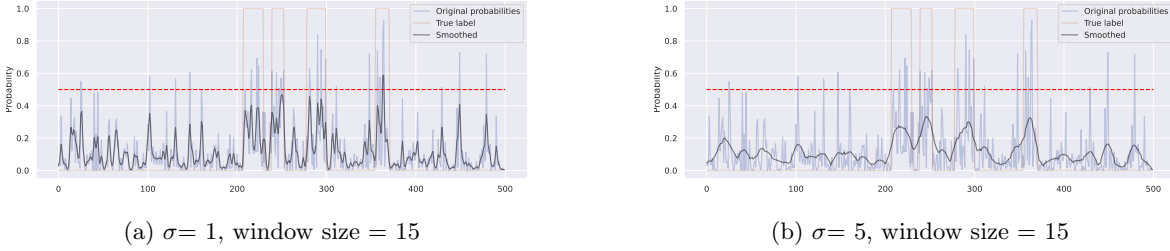


Figure 38: Gaussian averaging on 500 samples

We performed cross-validation to determine the optimal hyperparameters. As observed, smoothing generally lowers predicted probabilities, which can reduce the F1 score for some configurations. In sequential averaging, the F1 score may initially improve, as some class 0 instances previously misclassified as class 1 are corrected.

With a moving average window, the F1 score drops significantly because many probabilities fall below the 0.5 threshold. A similar trend is observed with Gaussian smoothing, although it is less sensitive to the window size due to the standard deviation. The best smoothing observed were obtained with Gaussian smoothing (window size = 15, $\sigma = 5$).

After our third [submission](#) using a transformer model for the true test set, we wanted to also smooth the probabilities to see if it improved classification. Since Gaussian smoothing flattens probability distributions and the threshold for the Presidents class is 0.5, we tested methods to rescale probabilities (sigmoid, softmax), but without success. This led us to consider using a model to adapt the threshold dynamically.

We first tried logistic regression to recalibrate probabilities, but the results were unstable (probabilities of 0.14 or 0.5) and did not capture temporal dependencies. We therefore turned to an RNN, which better models sequential patterns and leverages previous predictions, using a similar architecture as described in [Recurrent Neural Networks](#).

To enable the model to learn effectively, we preserved the order of the labels and fed the LSTM blocks with both probabilities and true labels. By varying the block size, we selected a size of 30, which accounts for the median of 26 in label continuity, as shown in Figure 31. When testing on the probabilities of the test set, we obtained satisfactory results, as illustrated in Figure 39.

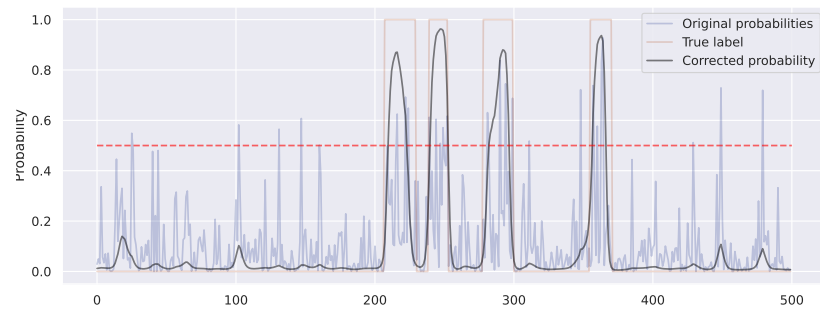


Figure 39: Corrected probabilities by LSTM on 500 samples

4 Submissions

We choose our best models for our classification task on the true test set.

4.1 Presidents

	F1	AUC	AP
Submission 1	12.80	73.34	36.89
Submission 2	24.27	50.40	14.18
Submission 3	60.96	91.56	74.79
Submission 4	82.11	98.17	93.48
Submission 5	73.36	98.17	92.08

Table 9: Presidents scores.

Notes: Submissions **1** and **2** should not really be considered, there was a mistake with the probabilities, where classes were mixed up, and a probability higher than 0.5 corresponded to Chirac and not Mitterrand. They were also submitted at the beginning of our experimentation when we tested different models and not chosen among our best models.

Submission 1

The first submission used a Logistic Regression with TF-IDF and preprocessing [Prep.3](#). When doing a cross-validation with this model, we had good F1 score, but those metrics didn't take account of the imbalance of the class (macro F1), explaining why we obtained low F1 score on the submission. As the metrics seemed good, we went to test this model as our first submission.

Submission 2

We submitted this after first testing a transformer model. We used the [distilled-bert-case](#) for tokenization and train. Not only the model isn't trained for french text, but we did not do early stopping, and used the overfitted model from the last epoch to make the predictions, resulting in the poor metrics.

Submission 3

For this submission, we finetuned the transformer model CamemBERTa as it showed the best results in section [Presidents](#). As we also suppose that in the test set, the true classes were continuous in blocks, we also wanted to smooth to the predicted probabilities using the Gaussian smoothing. Figure 40 show the predictions using the transformers Gaussian smoothed, but because of how the gaussians became flat, we didn't applied the smoothing.

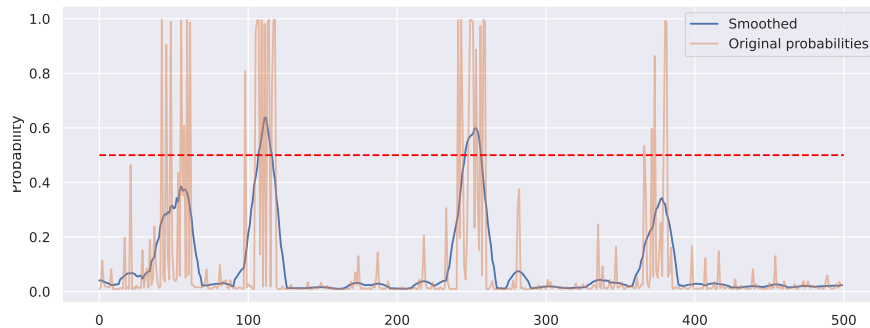


Figure 40: Gaussian smoothing window size = 15, $\sigma = 10$, on 500 samples

Submission 4

For this submission, we used the probabilities of submission 3, and corrected the probabilities using the LSTM model, as we have tested in Figure 39.

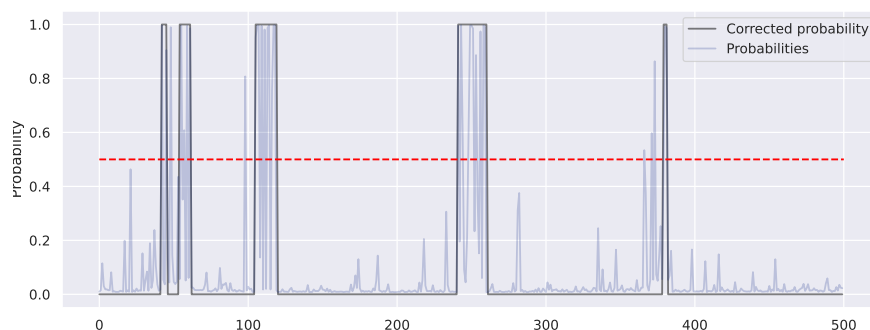


Figure 41: LSTM correction on 500 samples

We can notice that compared to the gaussian smoothing, if we could mix the 2 corrected probabilities, that would yield better results. We first tried to train the LSTM on the smoothed probabilities.

Submission 5

By training our LSTM on the gaussian smoothed probabilities, we made the predictions for this submission. As the results show, these smoothed probabilities alone are not enough, as we loose in Figure 42 some class 1 predictions, hence a mix of the two predictions should be considered.

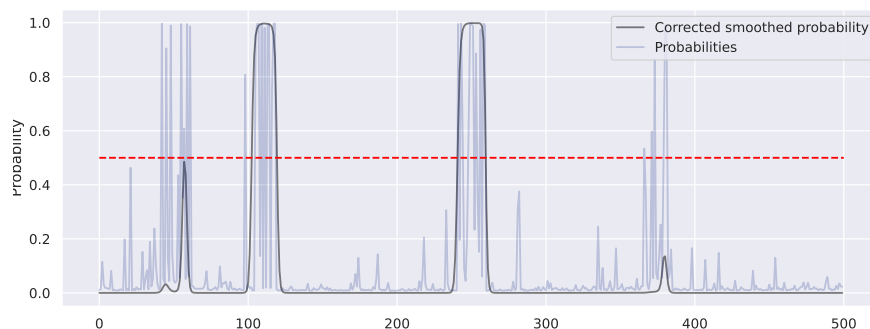


Figure 42: Corrected smoothed probability by LSTM on 500 samples

4.2 Movies

	F1	Accuracy	Precision	Recall
Submission 1	91.27	90.82	87.021	95.96
Submission 2	88.22	88.36	89.33	87.13
Submission 3	78.26	72.39	64.54	99.35

Table 10: Movies scores.

Submission 1

For this submission, we used the fine-tuned transformer model RoBERTaV1 since it was our best model yet. It has a token length of 512, batch size of 8, and learning rate of $2e^{-5}$. There is a very light weight decay (0.01).

Submission 2

For this submission, we used the GRU encoder-based seen in Figure 27 that was trained over 100 epochs.

Submission 3

This submission was an error. Instead of using the RoBERTaV3 (which we were curious about since it had a higher recall in our test data), we used the failed RoBERTaV5 by mistake, which explains the bad results.

5 Appendix

5.1 RNN

Here we put the scores and plots for the LSTM models.

Test metrics	F1	AP	AUC
LSTM POS tag	0.39	0.36	0.74
LSTM BoW	0.61	0.33	0.70
LSTM TF-IDF	0.51	0.13	0.49
LSTM encoder-based	0.50	0.50	0.82

Table 11: President LSTM test scores

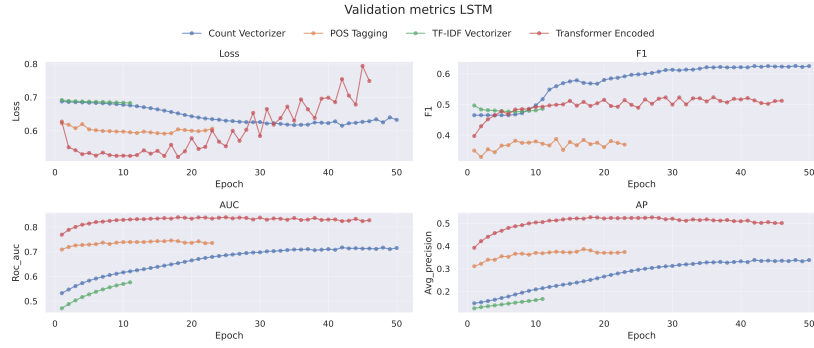


Figure 43: President LSTM metrics over epochs.

Test scores	F1	Accuracy	AUC
LSTM POS tag	0.64	0.52	0.57
LSTM BoW	0.67	0.50	0.53
LSTM TF-IDF	0.64	0.60	0.68
LSTM encoder-based	0.74	0.76	0.84

Table 12: Movies LSTM test scores

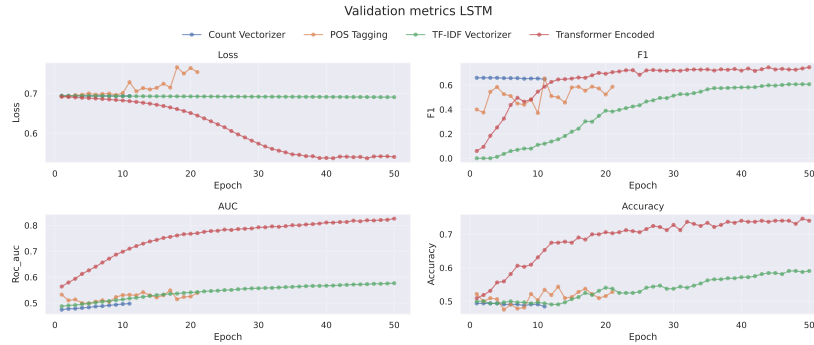


Figure 44: Movies LSTM metrics over epochs.